Lecture 8

BACKTRACKING

Backtracking

- Suppose you have to make a series of *decisions,* among various *choices,* where
  - You don't have enough information to know what to choose
  - Each decision leads to a new set of choices
  - Some sequence of choices (possibly more than one) may be a solution to your problem
- Backtracking is a methodical way of trying out various sequences of decisions, until you find one that "works"

Introduction

- Backtracking is used to solve problems in which a sequence of objects is chosen from a specified set so that the sequence satisfies some criterion.
- Backtracking is a modified depth-first search of a tree.
- Backtracking involves only a tree search.
- Backtracking is the procedure whereby, after determining that a node can lead to nothing but dead nodes, we go back ("backtrack") to the node's parent and proceed with the search on the next child.

Introduction …

- We call a node nonpromising if when visiting the node we determine that it cannot possibly lead to a solution. Otherwise, we call it promising.
- In summary, backtracking consists of
  - Doing a depth-first search of a state space tree,
  - Checking whether each node is promising, and, if it is nonpromising, backtracking to the node's parent.
- This is called pruning the state space tree, and the subtree consisting of the visited nodes is called the pruned state space tree.

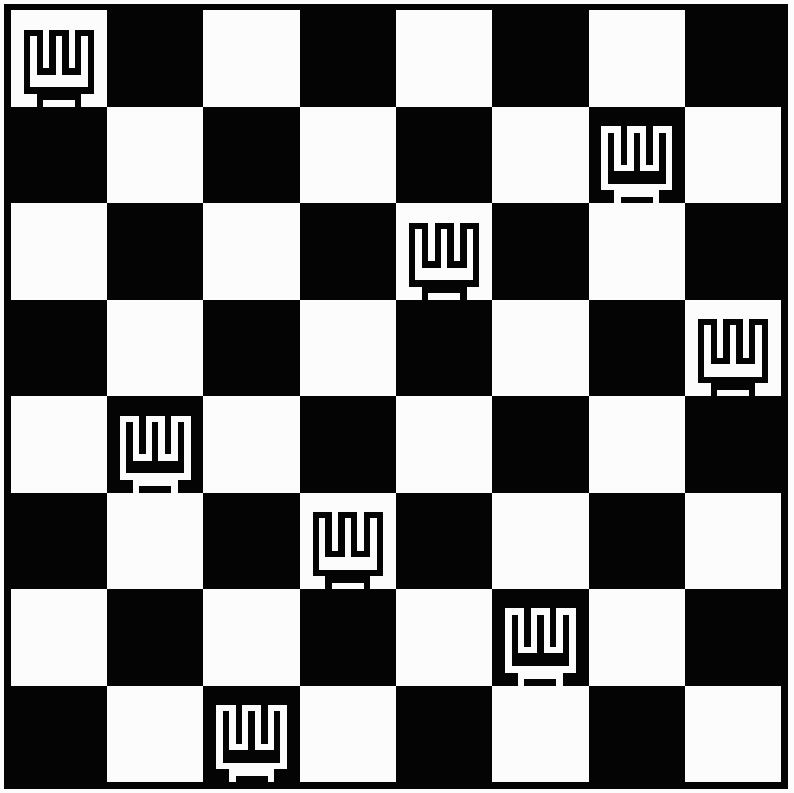
Backtracking – 8 Queens Problem

- Find an arrangement of **8** queens on a single chess board such that no two queens are attacking one another.

- In chess, queens can move all the way down any row, column or diagonal (so long as no pieces are in the way).

    - Due to the first two restrictions, it's clear that each row and column of the board will have exactly one queen.

- This is also called the N Queens problem since we can solve this problem for any NxN board with N Queens as well.

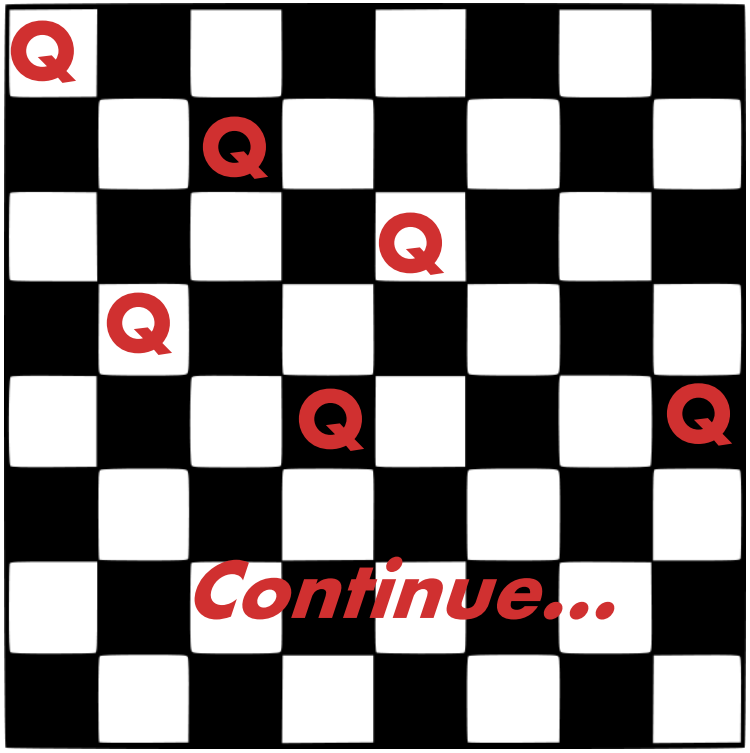


Backtracking – 8 Queens Problem

- The backtracking strategy is as follows:
  1) Place a queen on the first available square in row 1.

  2) Move onto the next row, placing a queen on the first available square there (that doesn't conflict with the previously placed queens).

  3) Continue in this fashion until either:
     a) you have solved the problem, or
     b) you get stuck.
        - When you get stuck, remove the queens that got you there, until you get to a row where there is another valid square to try.



Backtracking – Eight Queens Problem

- When we carry out backtracking, an easy way to visualize what is going on is a tree that shows all the different possibilities that have been tried.
- On the board we will show a visual representation of solving the 4 Queens problem (placing 4 queens on a 4x4 board where no two attack one another).

Backtracking in Decision Trees

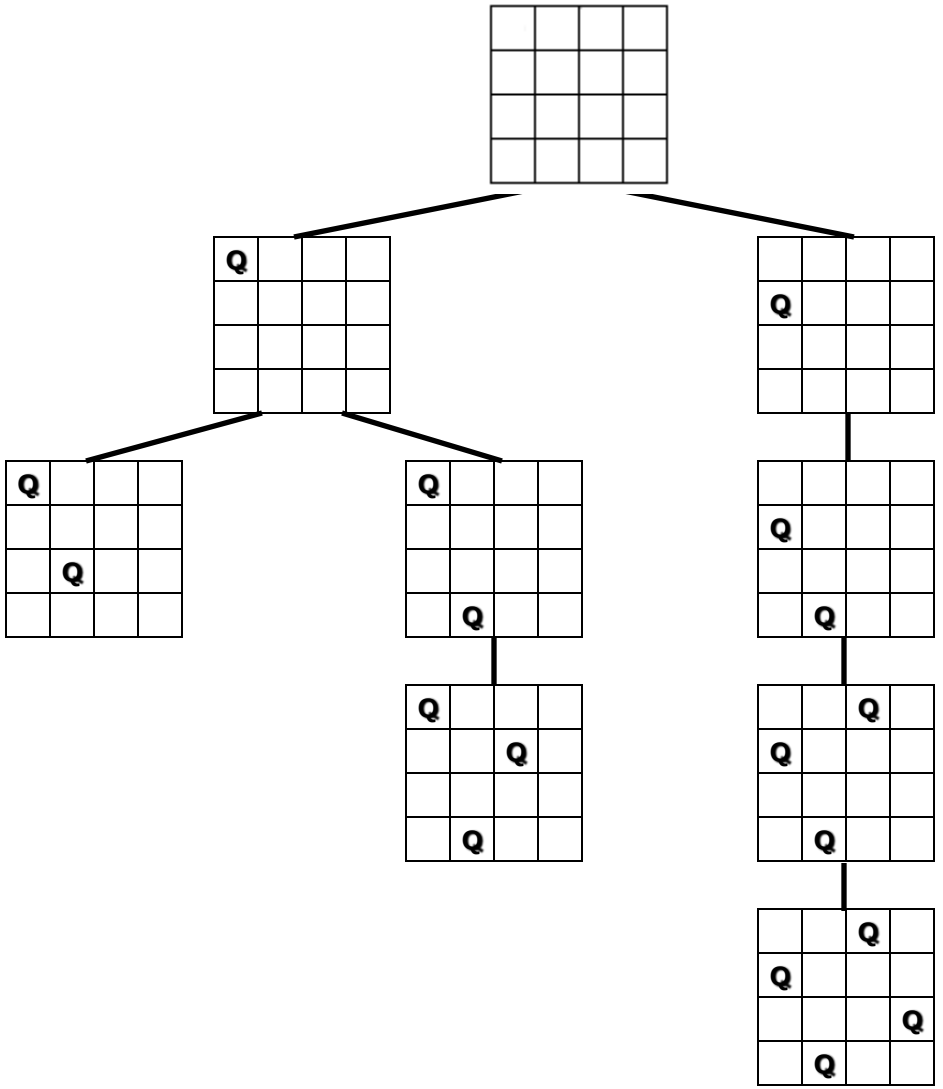
empty board

place 1st queen

place 2nd queen

place 3rd queen

place 4th queen



Backtracking – Eight Queens Problem

- The neat thing about coding up backtracking, is that it can be done recursively, without having to do all the bookkeeping at once.
  - Instead, the stack or recursive calls do most of the bookkeeping
  - (ie, keeping track of which queens we've placed, and which combinations we've tried so far, etc.)

perm[] - stores a valid permutation of queens from index 0 to location-1.
location – the row we are placing the next queen
usedList[] – keeps track of the columns in which the queens have already been placed.

```
void solveItRec(int perm[], int location, struct onesquare usedList[])  {

  if (location == SIZE) {
     printSol(perm);
  }
  for (int i=0; i<SIZE; i++) {

     if (usedList[i] == false) {

        if (!conflict(perm, location, i)) {

              perm[location] = i;
              usedList[i] = true;
              solveItRec(perm, location+1, usedList);
              usedList[i] = false;
           }
        }
     }
}
```

Found a solution to the problem, so print it!

Loop through possible columns to place this queen.

Only try this column if it hasn't been used

Check if this position conflicts with any previous queens on the diagonal

1) mark the queen in this column
2) mark the column as used
3) solve the next row location recursively
4) un-mark the column as used, so we can get ALL possible valid solutions.

Backtracking – 8 Queens Problem

- Animated Example:
- http://www.hbmeyer.de/backtrack/achtdamen/eight.htm#up

Example Problem

- Show the first 2 solutions to the 5 queens problem that this algorithm would create.
  - Also show the decision tree as shown in class.

Example Problem

- Finish the 5 Queens solution from this point, how many times do you have to backtrack?

Backtracking – 8 queens problem - Analysis

- Another possible brute-force algorithm is generate the permutations of the numbers 1 through 8 (of which there are 8! = 40,320),
  - and uses the elements of each permutation as indices to place a queen on each row.
  - Then it rejects those boards with diagonal attacking positions.

- The backtracking algorithm, is a slight improvement on the permutation method,
  - constructs the search tree by considering one row of the board at a time, eliminating most non-solution board positions at a very early stage in their construction.
  - Because it rejects row and diagonal attacks even on incomplete boards, it examines only 15,720 possible queen placements.

Another Example- Sudoku

- 9 by 9 matrix with some numbers filled in
- all numbers must be between 1 and 9
- Goal: Each row, each column, and each mini matrix must contain the numbers between 1 and 9 once each
  - no duplicates in rows, columns, or mini matrices

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Solving Sudoku – Brute Force

- A *brute force* algorithm is a simple but general approach
- Try all combinations until you find one that works
- This approach isn't clever, but computers are fast
- Then try and improve on the brute force resuts

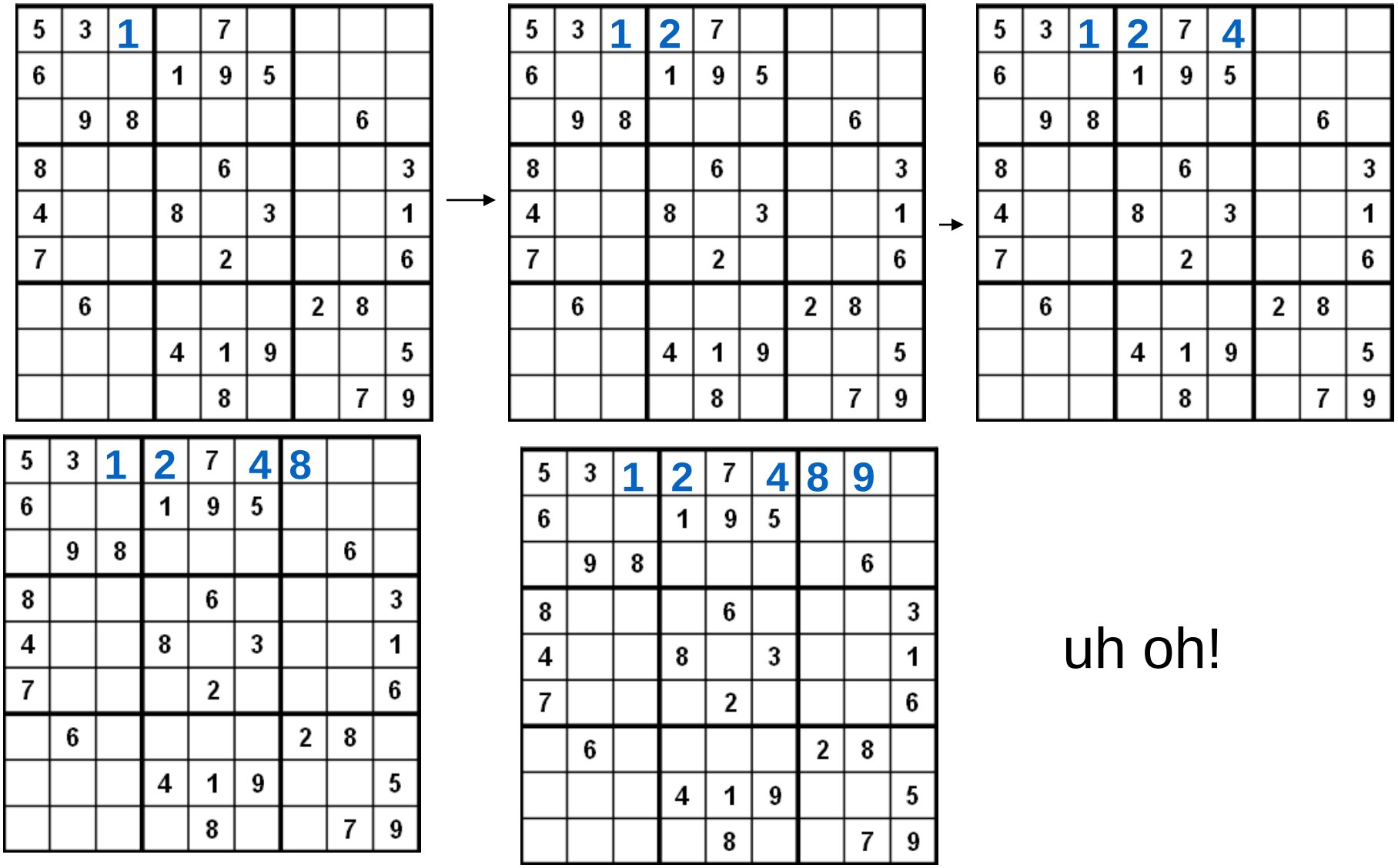
5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Solving Sudoku

- Brute force Sudoku Soluton
  - if not open cells, solved
  - scan cells from left to right, top to bottom for first open cell
  - When an open cell is found start cycling through digits 1 to 9.
  - When a digit is placed check that the set up is legal
  - now solve the board

5	3	1		7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Solving Sudoku – Later Steps



uh oh!

Sudoku – A Dead End

- We have reached a dead end in our search

5	3	1	2	7	4	8	9	
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

- With the current set up none of the nine digits work in the top right corner

Backing Up

- When the search reaches a dead end in ***backs up*** to the previous cell it was trying to fill and goes onto to the next digit
- We would back up to the cell with a 9 and that turns out to be a dead end as well so we back up again
  - so the algorithm needs to remember what digit to try next
- Now in the cell with the 8. We try and 9 and move forward again.

5	3	1	2	7	4	8	9	
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

5	3	1	2	7	4	9		
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Characteristics of Brute Force and Backtracking

- Brute force algorithms are slow
- The don't employ a lot of logic
  - For example we know a 6 can't go in the last 3 columns of the first row, but the brute force algorithm will plow ahead any way
- But, brute force algorithms are fairly easy to implement as a first pass solution
  - backtracking is a form of a brute force algorithm

Key Insights

- After trying placing a digit in a cell we want to solve the new sudoku board
  - Isn't that a smaller (or simpler version) of the same problem we started with?!?!?!?
- After placing a number in a cell the we need to remember the next number to try in case things don't work out.
- We need to know if things worked out (found a solution) or they didn't, and if they didn't try the next number
- If we try all numbers and none of them work in our cell we need to *report back* that things didn't work

- Problems such as Suduko can be solved using
- recursive because later versions of the problem are just slightly simpler versions of the original
- backtracking because we may have to try different alternatives

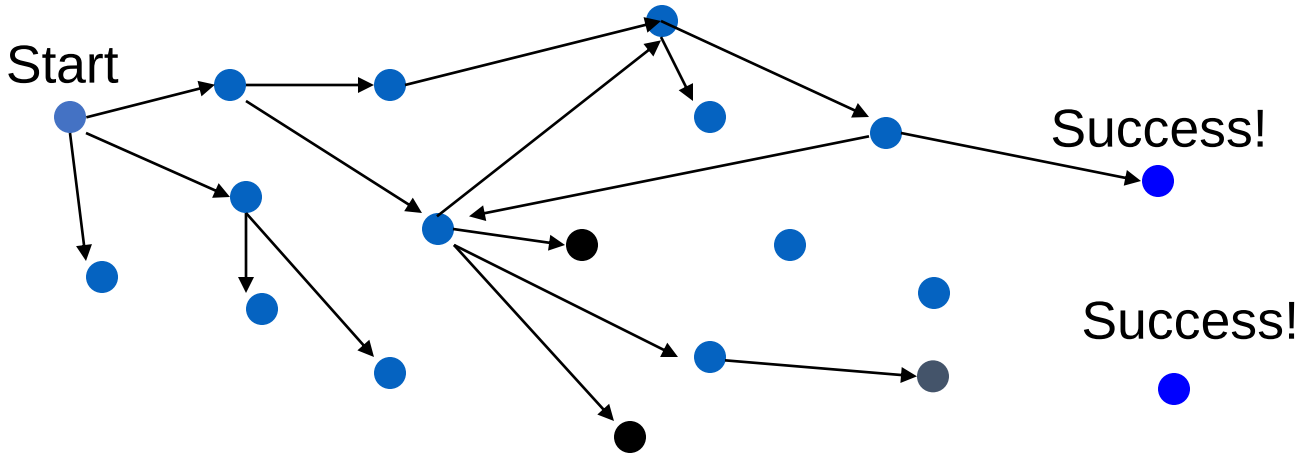
Pseudo code

```
bool SolveSudoku(Grid<int> &grid)
{
  int row, col;

  if (!FindUnassignedLocation(grid, row, col))
    return true; // all locations successfully assigned!

  for (int num = 1; num <= 9; num++) {    // options are 1-9
    if (NoConflicts(grid, row, col, num)) {  // if # looks ok
       grid(row, col) = num;                  // try assign #
       if (SolveSudoku(grid)) return true;    // recur if succeed stop
       grid(row, col) = UNASSIGNED;           // undo & try again
    }
  }
  return false;      // this triggers backtracking from early decisions
}
```

Sudoku Backtracking Animation

Goals of Backtracking

- Possible goals
  - Find a path to success
  - Find all paths to success
  - Find the best path to success
- Not all problems are exactly alike, and finding one success node may not be the end of the search

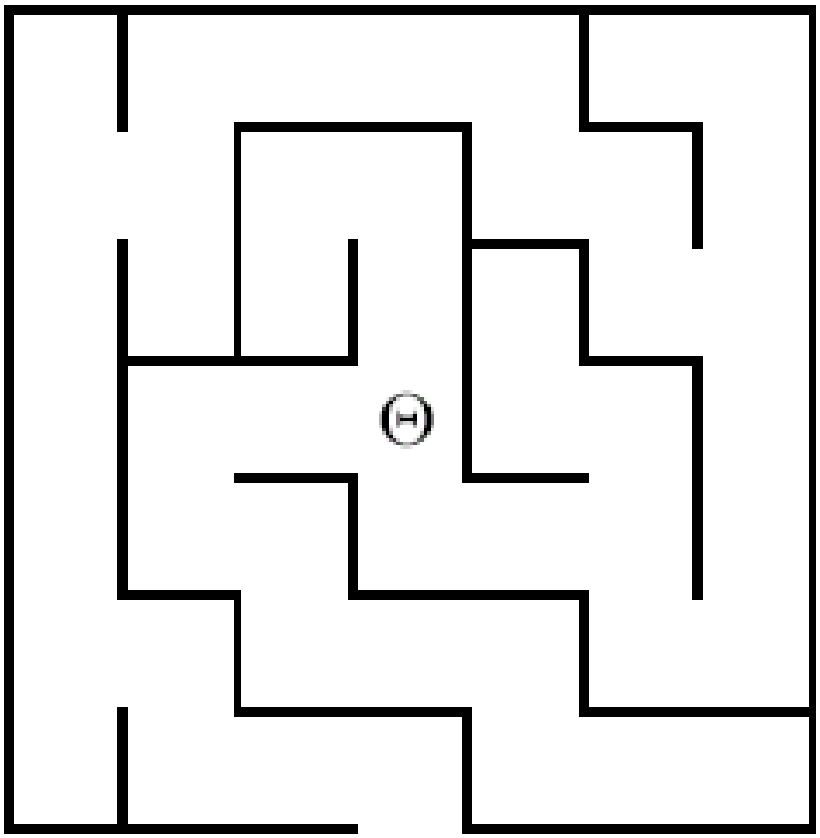


Mazes and Backtracking

- An example of something that can be solved using backtracking is a maze.
  - From your start point, you will iterate through each possible starting move.
  - From there, you recursively move forward.
  - If you ever get stuck, the recursion takes you back to where you were, and you try the next possible move.

- In dealing with a maze, to make sure you don't try too many possibilities,
  - one should mark which locations in the maze have been visited already so that no location in the maze gets visited twice.
  - (If a place has already been visited, there is no point in trying to reach the end of the maze from there again.

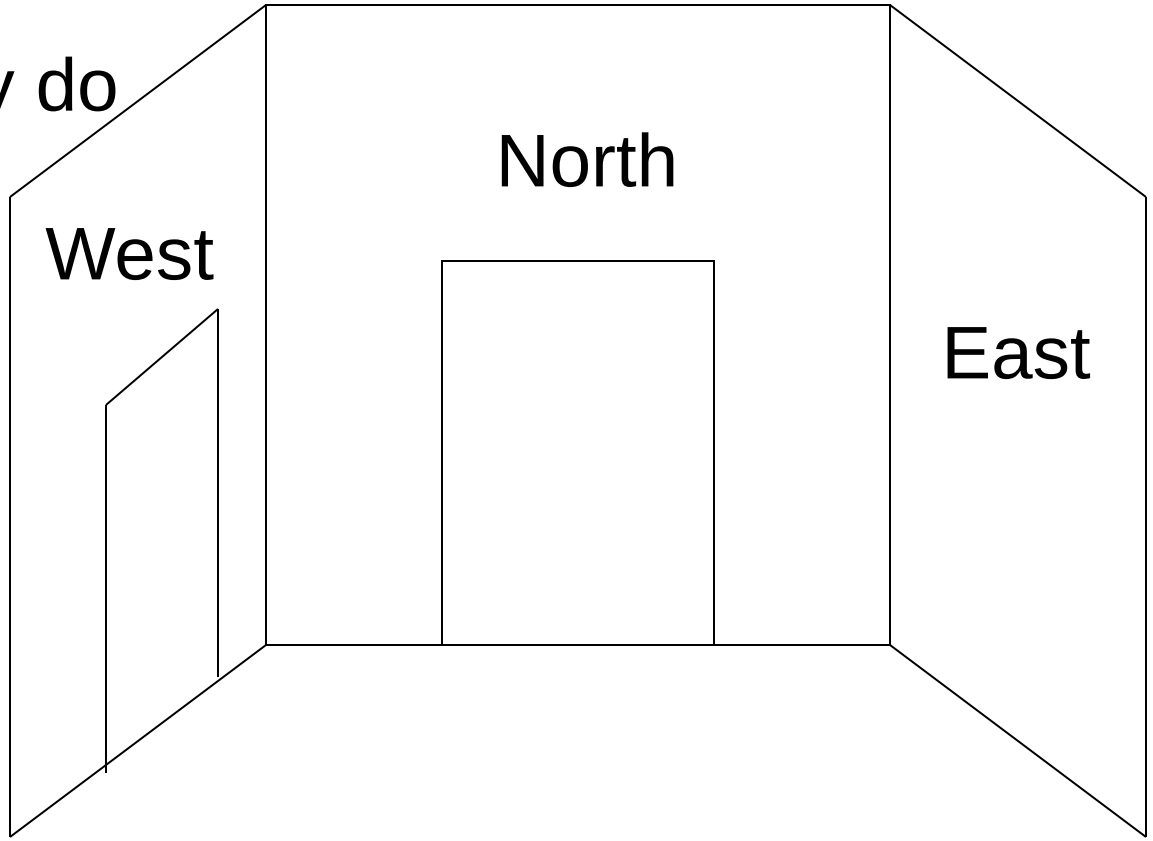
Another Backtracking Problem
A Simple Maze



Search maze until way out is found. If no way out possible report that.

The Local View

Which way do I go to get out?

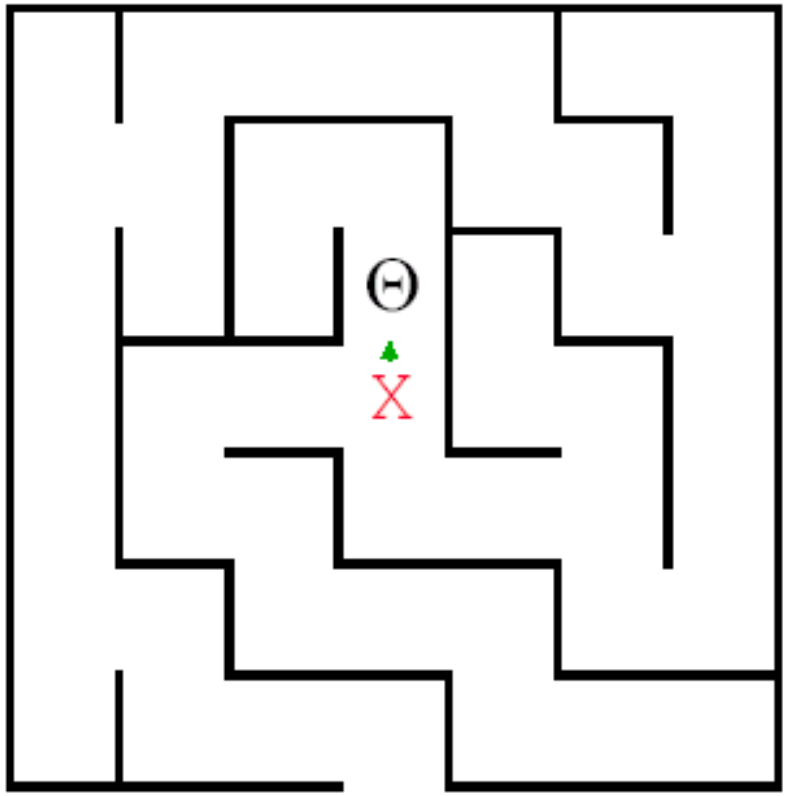


Behind me, to the South is a door leading South

Modified Backtracking Algorithm for Maze

- If the current square is outside, return TRUE to indicate that a solution has been found.
  If the current square is marked, return FALSE to indicate that this path has been tried.
  Mark the current square.
  for **(**each of the four compass directions**)**
  { if **(** this direction is not blocked by a wall **)**
  { Move one step in the indicated direction from the current square.
  Try to solve the maze from there by making a recursive call.
  If this call shows the maze to be solvable, return TRUE to indicate that fact.
  }
  }
  Unmark the current square.
  Return FALSE to indicate that none of the four directions led to a solution.

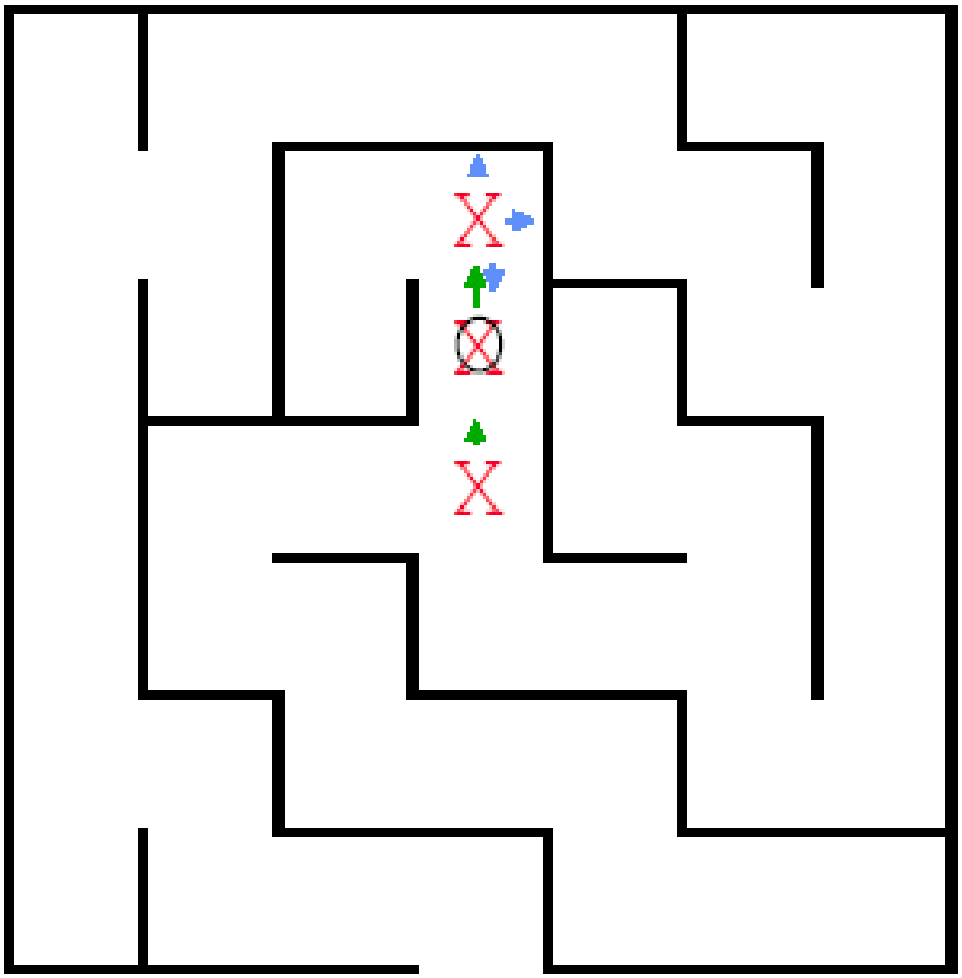
Backtracking in Action



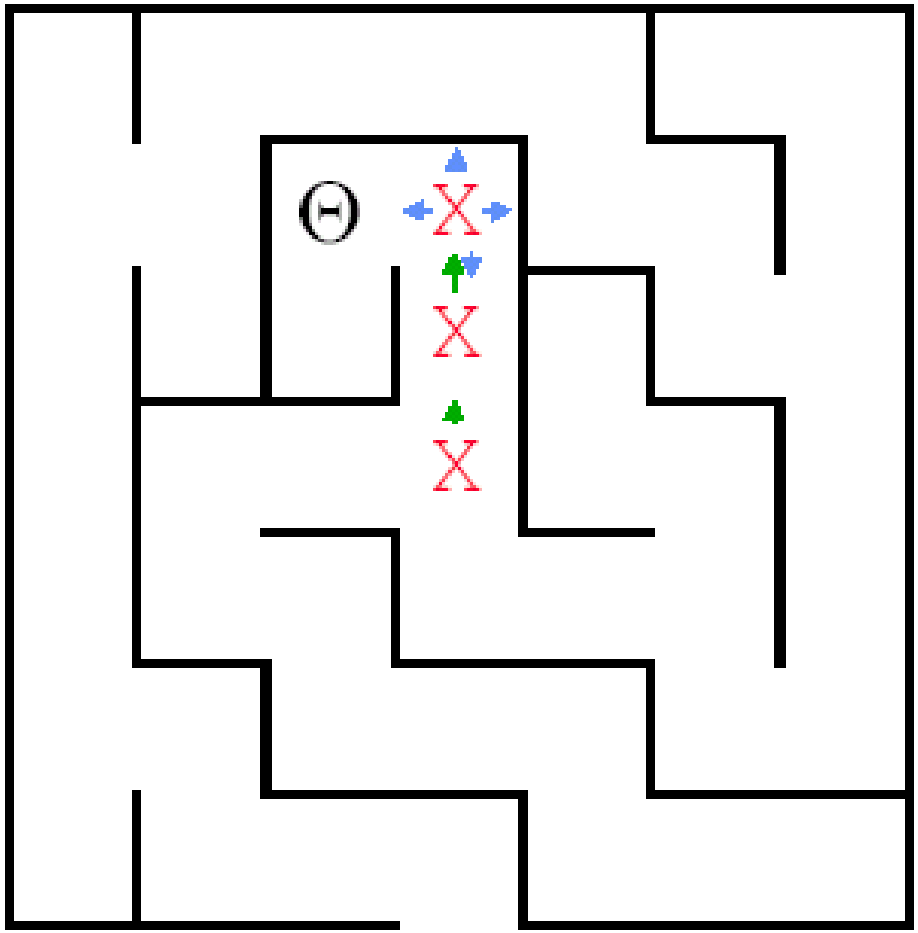
The crucial part of the algorithm is the for loop that takes us through the alternatives from the curren square. Here we have move to the North.

```
for (dir = North; dir <= West; dir++)
{      if (!WallExists(pt, dir))
       {if (SolveMaze(AdjacentPoint(pt, dir)))
              return(TRUE);
}
```

Backtracking in Action

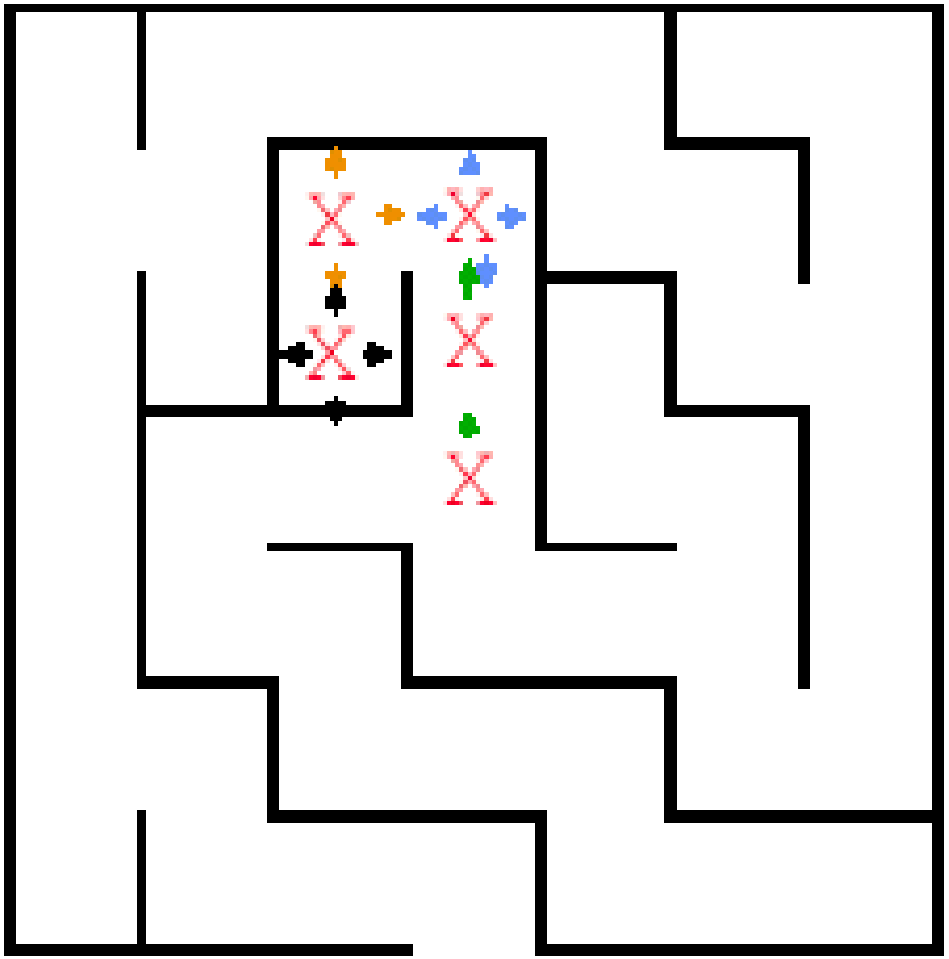


Here we have moved North again, but there is a wall to the North . East is also blocked, so we try South. That call discovers that the square is marked, so it just returns.



So the next move we can make is West.

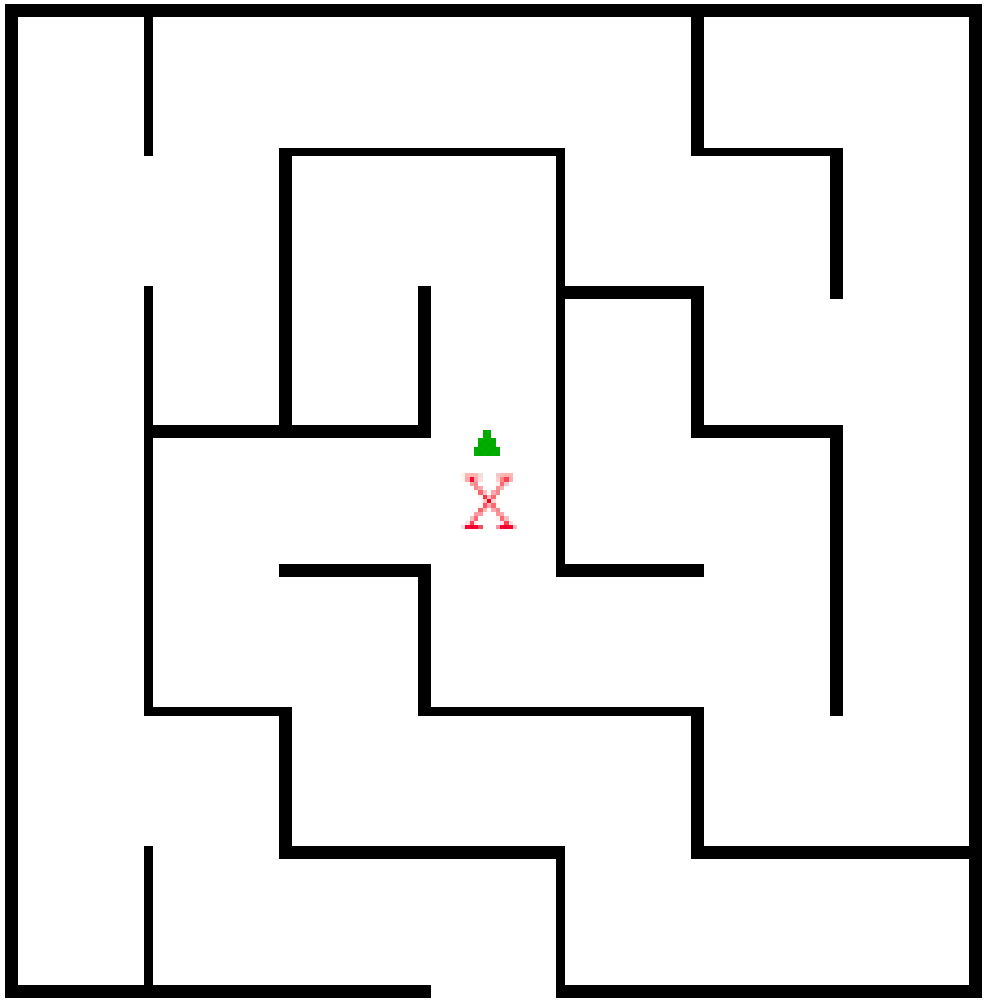
Where is this leading?



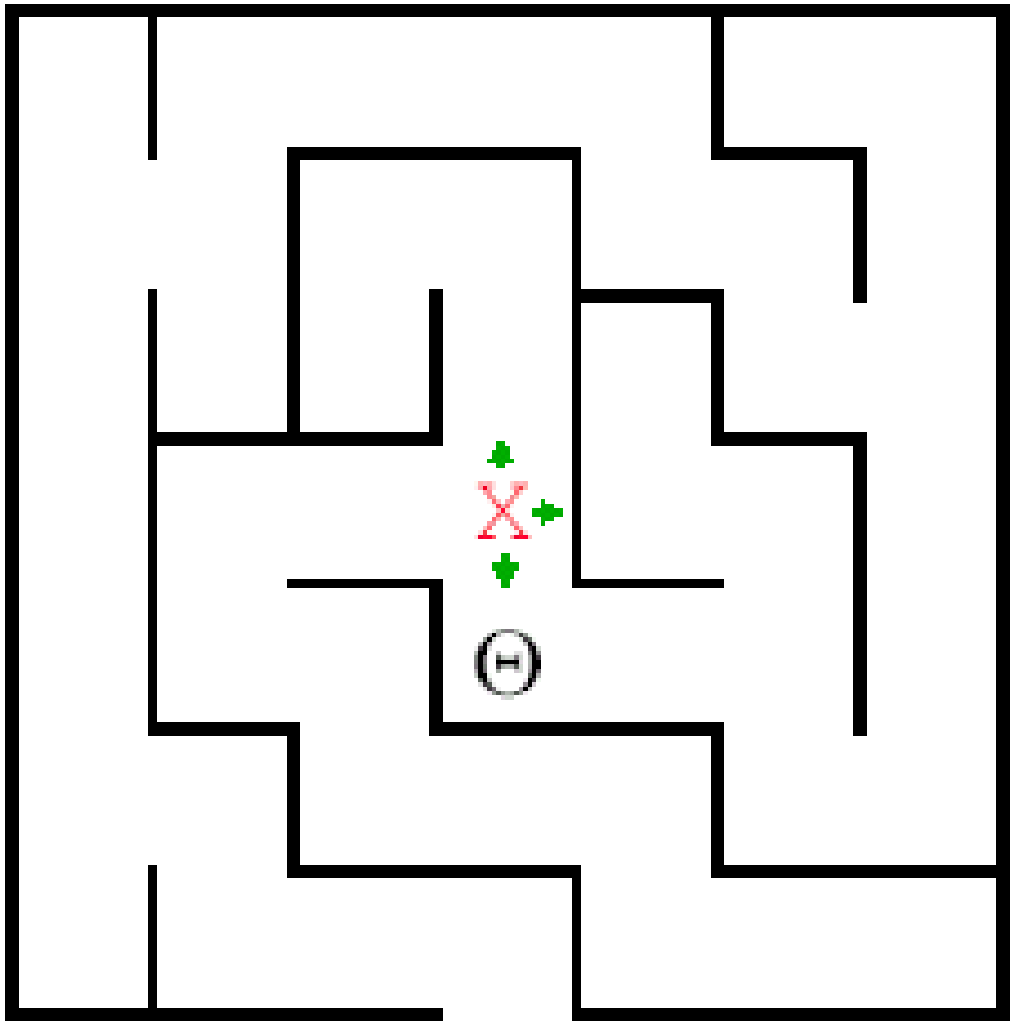
This path reaches a dead end.

Time to backtrack!

Remember the program stack!

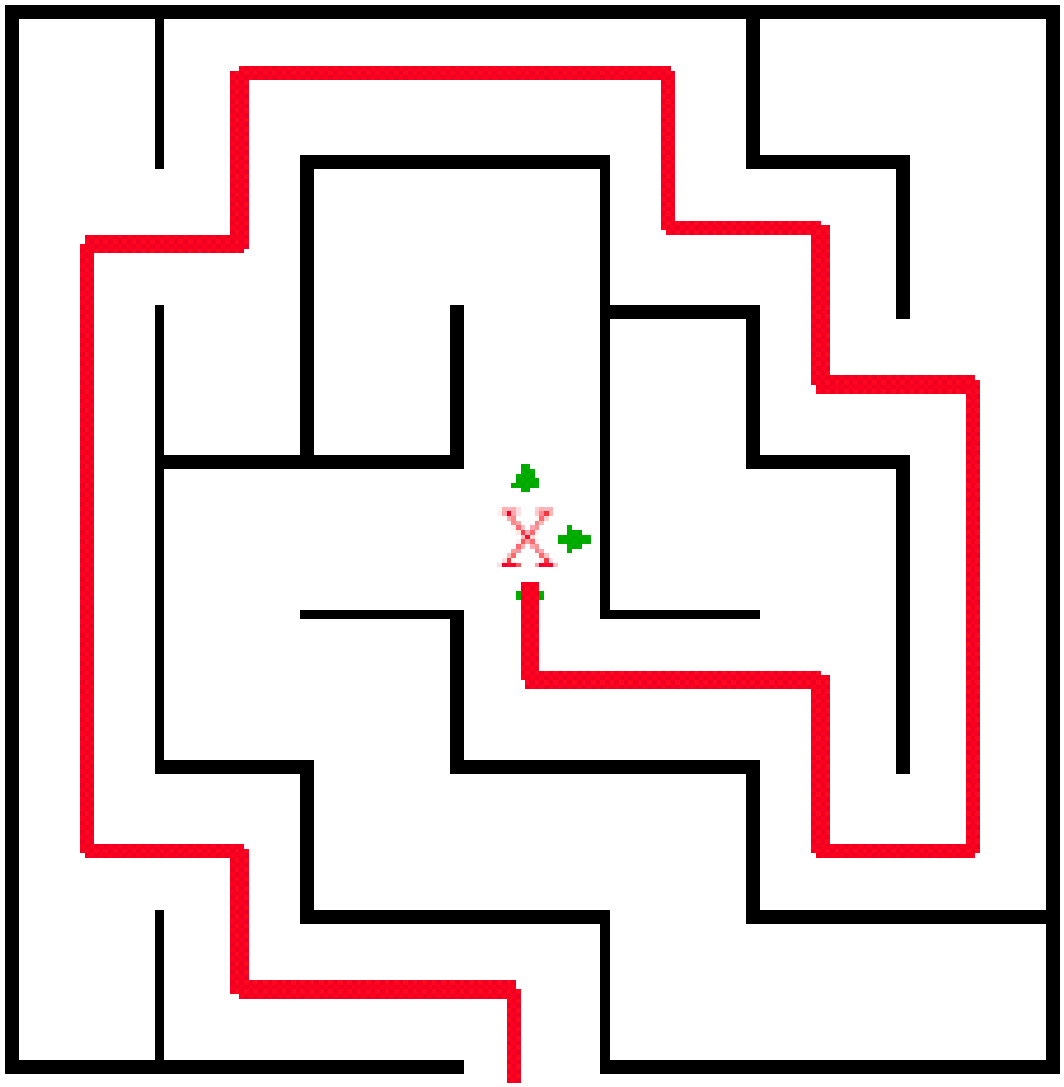


The recursive calls end and return until we find ourselves back here.



And now we try South

Path Eventually Found



Visualization and code

- https://trinket.io/python/6ec2ecb76b

Graph Coloring Problem

Let G be undirected graph and let c be an integer.

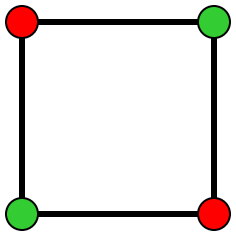
Assignment of colors to the vertices or edges such that no two adjacent vertices are to be similarly colored.

We want to minimize the number of colors used.

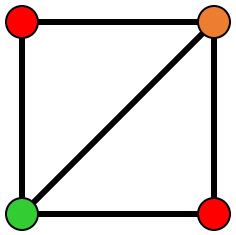
The smallest c such that a c-coloring exists is called the graph's chromatic number and any such c-coloring is an optimal coloring.

Coloring of Graph

- The graph coloring optimization problem: find the minimum number of colors needed to color a graph.
- The graph coloring decision problem: determine if there exists a coloring for a given graph which uses at most m colors.



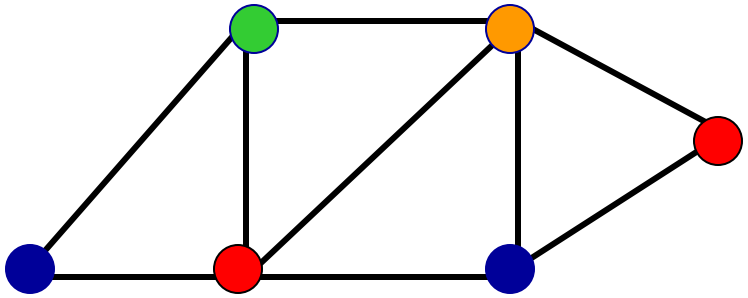
Two colors



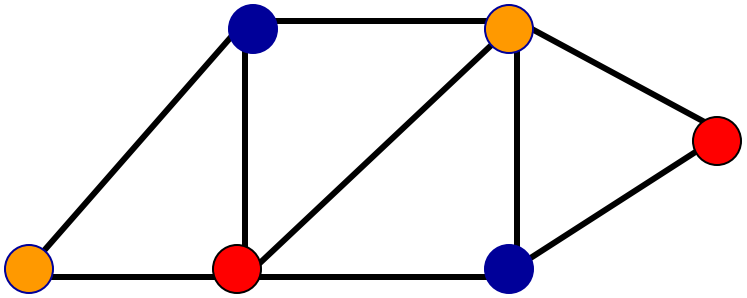
No solution with two colors

Coloring of Graphs

- Practical applications: scheduling, time-tabling, register allocation for compilers, coloring of maps.
- A simple graph coloring algorithm - choose a color and an arbitrary starting vertex and color all the vertices that can be colored with that color.
- Choose next starting vertex and next color and repeat the coloring until all the vertices are colored.



Four colors



Three colors are enough

Psudocode

```
Color(C,j,k,n)
      if j = n+1 then
           output C
           return or quit
     for i = 1 to k
           C[j] = I
           if valid(C,j,n) then
                      Color(C,j+1,k,n)

Valid(C,j,n)
      for all neighbors v of j with v < j
            if C[v] = C[j] then
                      return false
      return true
```

Subset Sum

- Given a set *X* of positive integers and *target* integer *T*, is there a subset of elements in *X* that add up to *T*?
- Notice that there can be more than one such subset.
- For example,
- if *X* = {8, 6, 7, 5, 3, 10, 9} and *T* = 15, the answer is True, because the subsets
- {8, 7} and {7, 5, 3} and {6, 9} and {5, 10} all sum to 15. On the other hand, if
- *X* = {11, 6, 5, 1, 7, 13, 12} and *T* = 15, the answer is False.

Subset Sum

- There are two trivial cases. If the target value *T* is zero, then we can immediately return True, because the empty set is a subset of *every* set *X*, and the elements of the empty set add up to zero.6 On the other hand, if *T* < 0, or if *T* 6= 0 but the set *X* is empty, then we can immediately return False.
- For the general case, consider an arbitrary element *x* 2 *X*. (We've already handled the case where *X* is empty.) There is a subset of *X* that sums to *T* if and only if one of the following statements is true:
  - There is a subset of *X* that *includes x* and whose sum is *T*.
  - There is a subset of *X* that *excludes x* and whose sum is *T*.

Subset Sum

```
SubsetSum(S,T,k,n)
if k = n+1 then
  output S
  return
S[k] = 0
if InRange(S,T,k,n) then
  SubsetSum(S, k+1, n)
S[k] = 1
if InRange(S,T,k,n) then
  SubsetSum(S, k+1, n)
```

```
InRange(S,T,k,n)
S1 = Sum_{i=1}^{k} T[i]S[i]
if S1 > M then
  return false
S2 = Sum_{i=k+1}^{n} T[i]
if S1 + S2 < M then
  return false
return true
```